Lecture 4 — An Agent That Improves a Number

Design note

15.773 Intensive Hands-On Deep Learning

MIT Sloan School of Management • Fall 2026 • Prof. Vivek Farias

We start with the shape of research itself, then build the four things a machine needs in order to do it, and finish by assembling them into the only artifact the agent ever actually reads: about thirty lines of text.

The whole lecture in one line. Give an agent a number and tell it to make the number better. What it does next is the subject.

1. How research gets done

1.1 The cycle

Strip research of its trappings and a loop is left. You have a question. You form a guess about what might improve the answer. You build something concrete enough to try. You measure it. And the measurement changes your next guess.

```
state <- what I know so far

repeat:
    hypothesis <- propose(state)        # what might make it better?
    experiment <- build(hypothesis)     # make it concrete enough to
        run
    result     <- measure(experiment)   # a number I did not get to
        choose
    state      <- state + (hypothesis, result)
```

The third line is what separates research from guessing. A result you chose is a decoration; a result you did not choose can contradict you. Much of this lecture is in service of making that line real — of building a measurement the agent genuinely cannot influence.

1.2 The same cycle, mechanised

Lecture 3 ended with a claim: an agent is a prompt in a loop. Nothing today changes that. What changes is what is in the agent's hands — and with the right two tools, the cycle above *is* the agent loop.

```
repeat:
    code    <- model(context)        # propose and build, in one step
    output  <- sandbox.run(code)     # stdout, or a traceback
    score   <- scorer.score()        # the measurement it does not
        control
    context <- context + [code, output, score]
```

Notice that *propose* and *build* have collapsed into a single step. For a model that writes programs, having the idea and expressing it are the same act — which is a large part of why any of this works.

1.3 The same cycle, as code

Close to what actually runs, with the bookkeeping stripped out.

```
def run_agent(goal, max_turns=60):
    messages = [{"role": "system", "content": PROMPT},
                {"role": "user",   "content": goal}]

    for turn in range(max_turns):
        reply = client.chat.completions.create(
            model=MODEL, messages=messages, tools=TOOLS
        ).choices[0].message

        if not reply.tool_calls:            # asked for nothing -> it
            is done
            return reply.content

        messages.append(reply)              # what it decided to do

        for call in reply.tool_calls:
            args   = json.loads(call.function.arguments)
            result = TOOL_FUNCTIONS[call.function.name](**args)
            messages.append({"role": "tool",
                             "tool_call_id": call.id,
                             "content": str(result)})
```

There is no research in it. No planning, no hypothesis, no decision about what to try next. Every one of those happens inside `client.chat.completions.create`. The function around it is a courier.

The two `append` lines are the whole mechanism. Strip them out and the model begins again from nothing every turn, with no memory of what it tried or what happened. Put them back and you have an agent.

> The loop is a **context-accumulation machine**, and the accumulation is the intelligence.

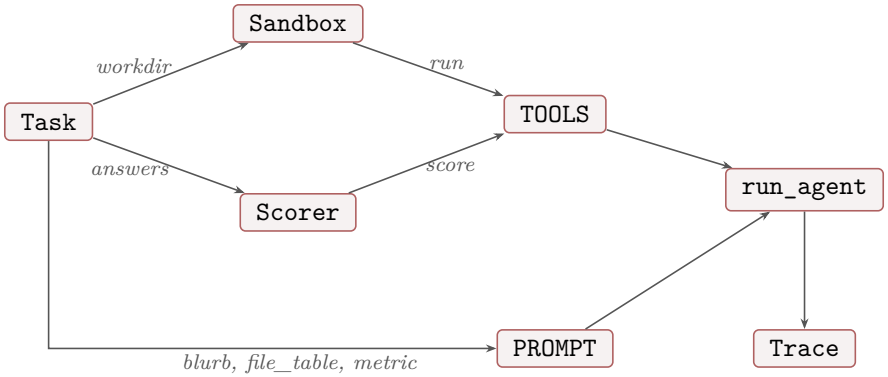
1.4 What is ours to design

Three things, and only three: **what tools exist**, **what goes into the context**, and **what ends the loop**. Everything else is downstream of those.

The third is worth sitting with. The loop exits when the model stops asking for tools — nobody wrote a rule about when a research project is finished. `max_turns` is a safety belt, not a design parameter. Whether that is a sensible way to end a piece of research is one of the questions this lecture exists to ask.

2. What the loop needs

Four things, in the order the loop needs them: a **problem**, a definition of **better**, a **measurement**, and somewhere to **run experiments**.



The one edge not drawn: `Sandbox` *also contributes its execution model and package list to* `PROMPT`*, so the instructions always describe the environment the agent is really in.*

2.1 `Task` — what is being asked

Abstractly. A held-out prediction problem. One table of observations, split three ways, with some columns visible at prediction time and some not. You declare five things:

`keys`	columns that identify a row — present in training data *and* at prediction time
`features`	columns usable at prediction time
`train_only`	columns observed historically but unavailable when predicting
`target`	the column to predict
`metric`	how a prediction is judged

Everything else follows mechanically. The files are *derived* from those roles:

```
train.csv        =  keys + features + train_only + [target]
dev_pairs.csv    =  keys + features
test_pairs.csv   =  keys + features
predictions.csv  =  keys + [prediction]
```

> The gap between the first line and the second **is** the task. It is not written anywhere; it is the arithmetic difference between two column lists. Move a column from `train_only` to `features` and the problem changes, the files change, and the prompt changes — without touching any prose.

Then the split: one frame, a seeded permutation, 80/10/10. The training answers go into the file. The dev and test answers stay in memory and are never written anywhere at all.

The **seed is the secret**. Not the data — the data is public. What the agent cannot reconstruct is *which rows landed in dev*, and that alone is enough to make the held-out set genuinely held out.

Concretely.

```
keys       = ["row_id", "col_id"]    # a user and a movie
features   = []                      # none at prediction time
train_only = ["t"]                   # timestamps, train only
target     = "value"                 # the rating, 1-5
metric     = rmse(clip=(1, 5))
```

Three rows of each file, side by side:

```
what the agent is asked:        what we are holding back:
 row_id  col_id                  row_id  col_id  value
   3153    3599                    3153    3599      4
   2690    3167                    2690    3167      3
   3412    3095                    3412    3095      4
```

Two bare integers in, a rating out. There is nothing else in the input, which is exactly why the task pushes the agent toward matrix completion: the only signal available is the pattern of *other* ratings by the same row and of the same column.

Why `row_id` and not `user`. Underneath this is MovieLens 1M — *the* standard benchmark for rating prediction — so any competent model has read papers about it and can recall that matrix factorization works well without discovering anything. The task is therefore **anonymised**: neutral column names, reshuffled identifiers, no titles or genres, and a row count that is not the recognisable 1,000,209. The *structure* survives on purpose. Reasoning from a sparse table of integers to matrix completion is generalisation; reciting a leaderboard is not. The switch is what separates them.

2.2 `Metric` — what "better" means

Abstractly. A name, a function, a direction, and optionally a clamp. The function is the obvious part. The *direction* is the one that is easy to underrate: `lower_is_better` silently controls which attempt counts as an improvement worth keeping, whether the leaderboard runs a running minimum or a maximum, what "best so far" means, and which way the gap between dev and test points.

Get it wrong and the closing number of the lecture prints backwards. It did, until the harness was pointed at an accuracy task and reported a *better* test score as evidence of optimism.

The clamp belongs to the task rather than to the metric family — RMSE does not know that ratings stop at 5 — so it is a parameter, not a property.

Concretely. `rmse(clip=(1,5))`. Root mean squared error, lower is better, predictions clamped to the legal rating range before scoring. The clamp is a small kindness: a model that predicts 5.4 for an enthusiast is not wrong in a way worth punishing.

2.3 `Scorer` — the measurement, and the deliverable

Abstractly. Two jobs. The second one is a repair.

The number. Read the agent's predictions, line them up against the held-out answers by

`keys`, refuse if any row is unpredicted, apply the metric. One float. This is the agent's only sense organ; it has no other way to learn whether an idea was any good.

This object is also the **trust boundary**. It is the only thing holding both the answers and the ability to read the agent's output, so the asymmetry is enforced by what each object *has* rather than by rules anyone has to remember.

> `score` is a *tool call*, so the agent cannot invoke it from inside its own Python. One configuration per turn. That rules out hyperparameter sweeps — and enforces, by accident, the discipline of changing one thing at a time and measuring it. One run, denied sweeps, built itself an internal validation split so it could tune offline and spend its single scoring call on the winner. That was the best thing any run did, and nobody told it to.

The deliverable. Every attempt writes predictions for the dev pairs *and* the test pairs. Whenever an attempt turns out to be the best so far, its test predictions are copied somewhere the agent cannot reach. The agent never decides when to submit.

That job exists because the obvious alternative failed twice. Asking the agent to write its final answer *"when you are done"* meant one run never judged itself done, ran out of turns and produced nothing at all — and another lost track of which model had been its best and shipped a worse one, reporting 0.8579 while submitting a 0.8616 model.

> Coupling the deliverable to the stopping decision was the mistake. Uncoupling them makes both failures impossible to express.

Why there is a dev set *and* a test set. This is the idea the lecture closes on. Every time the agent reads a dev score and chooses what to try next, it spends a little information about dev. Do that twenty-eight times and dev has been *fit* — not by gradient descent, but through a sequence of keep-this, discard-that decisions. Each one is a parameter update made of judgement instead of derivatives.

> The agent is playing the role the optimiser usually plays, and it overfits the validation set exactly the way an unconstrained hyperparameter search does. Test informed no decision, so test is the honest number.

What the last cell prints:

```
best dev RMSE          0.9815   (over 1 attempts)
submitted from       attempt 1, which scored 0.9815 on dev
test RMSE              0.9843
optimism              +0.0028  (test is worse than dev)
```

2.4 `Sandbox` — where experiments happen

Abstractly. A Python process the agent can send code to, with the task's files as its working directory. Most of it is plumbing. One decision in it is not.

Whether the process **persists between calls** sets the price of a marginal experiment — and therefore how much research gets done.

With a fresh process each time, every attempt pays full freight: reload the data, rebuild the model, retrain from zero. With a session that stays alive, *"train five more epochs on the model I already have"* costs three lines and a couple of seconds. The same agent will do different research depending on which one it has.

Timeouts follow from the same reasoning. The obvious implementation — kill the process when a step runs long — destroys every variable, which punishes precisely the long training runs the task is supposed to reward. So the interrupt happens *inside* the session: a timer fires, the step stops, the agent is told, and its variables survive.

The package list is **probed, not declared**. Colab and a laptop differ, and a prompt that lies about the environment wastes turns.

Concretely. `Sandbox(task.workdir, persistent=True, timeout=180)` — three minutes per step, enough to fit a factorization on 800k ratings and not enough to sit through a bad idea. It reports `numpy, pandas, scipy, scikit-learn`, because that is what it found.

3. The prompt, assembled

Everything above exists to produce one artifact: the text the agent actually reads. Here it is, in order, with every piece traced back to where it came from.

/ **blue** from the `Task` / **green** from the `Sandbox` hand-written

Shaded blocks are generated wholesale. Coloured words are values dropped into a sentence that is otherwise ours.

```
You are a researcher. You are given a sparse table of observations
indexed by two identifiers.
```

↩ *`task.blurb` — the one sentence written per dataset. This is where anonymisation lives or dies: say "movie ratings" here and the whole experiment changes.*

```
Your job is to predict the missing `value` for each held-out row, as
accurately as possible. The metric is RMSE. Lower is better.
```

↩ *`task.target`, then `task.metric.name` and `task.metric.direction` — §2.2. The direction is printed, not assumed.*

```
Your working directory contains:

  train.csv       row_id,col_id,t,value  (797,766 rows)  - observed data
  dev_pairs.csv   row_id,col_id          (99,720 rows)   - predict value for each
  test_pairs.csv  row_id,col_id          (99,722 rows)   - predict these at the very
      end
```

↩ *`task.file_table()` — computed from the column roles in §2.1. The agent is handed the shape of its own information gap, because that gap is just the difference between two of these lines.*

```
- Each `value` is an integer from 1 to 5.
- `t` is a time index.
- `t` appears only in train.csv, not in the pairs files, so it cannot
  be used at prediction time.
```

↩ *`task.notes_block()`. The first two come from `notes=[...]`; the third is **generated** from `train_only`. An earlier version omitted it, and an agent burned a whole attempt building a time model before discovering the column was not there.*

```
You have a persistent Python session: variables, loaded data and trained
models stay alive between run_python calls, so load the data once and
build on what you already have. Any single step that runs longer than
180s is stopped and handed back to you as a timeout - your variables
survive, so pick up where you left off and do less per step.

Available packages: numpy, pandas, scipy, scikit-learn.
```

↩ *`sandbox.description()` — §2.4. Both halves are facts about the object rather than claims about it: flip `persistent` and this paragraph rewrites itself, and the package list is whatever the child process reported when asked.*

```
For each attempt:
  1. Use run_python to build a model, then write BOTH of these, each
     with columns row_id,col_id,prediction:
        predictions.csv       one row per row of dev_pairs.csv
        test_predictions.csv  one row per row of test_pairs.csv
  2. Call score. It scores your dev predictions, and whenever an attempt
     is your best so far it keeps that attempt's test predictions as
     your submission.
  3. Use the number to decide what to try next.
```

↩ *The cycle from §1.1, made operational. The column list is `task.prediction_columns`; the rest is the `Scorer` contract from §2.3 written out by hand.*

```
You cannot see the true dev values. score is the only way to learn how
you are doing.

You never have to decide when to submit - your best attempt is saved
automatically, so there is no final step to remember and nothing is
lost if you run out of turns.
```

↩ *Hand-written, but describing the two design decisions from §2.3. The second sentence is the repair — it is what makes "ran out of turns and delivered nothing" impossible.*

```
Guidance:
- Start with the simplest thing that works, measure it, and improve
  from there.
- Change one thing at a time, so you know what caused a change in
  the score.

Stop when you are no longer making progress, and report what you tried
and what worked.
```

↩ *Hand-written. Note that none of it is about* this *problem; the same lines would serve any task at all.*

3.1 Two things to notice

Every word that depends on the dataset is coloured; everything left in grey is dataset-independent. Swap the `Task` and the prompt rewrites itself. There is no prose to keep in sync, and no way for the instructions to end up describing the previous problem.

And what it does not say. Nothing about *how* to predict a value. No baselines, no bias terms, no factorization, no hint that a ladder of standard approaches exists at all. The prompt describes the situation and the rules, and then stops. Whether anything gets climbed is the agent's business — which is the only way the question we are asking stays an open one.

4. Where this goes

The machinery is a vehicle for one question: *what does an agent do when you give it a number and tell it to make the number better?*

The short answer, across the runs recorded in `Supporting Material/Runs/`, is that it climbs a little way and then stops — usually well short of what its own approach could have given it, sometimes without ever trying the structural change that would have mattered, and with enormous variance between runs of an identical configuration. One run never found matrix factorization at all. Another found it and under-trained it. A third kept going until it exhausted its budget and left nothing behind.

That is the lecture. Everything above exists to make it measurable.